

Universidad Nacional de Colombia

Estructuras de Datos

Uso de estructuras de datos lineales en un juego de póker

Documento explicativo – Primera entrega

Integrantes

David Santiago Montiel Rodríguez

David Santiago Toro Arias

Jean Carlo Triana Guzmán

Docente

Jonatan Gómez Perdomo

22 de octubre de 2026

1. Introducción

Este proyecto consiste en un juego de póker Texas Hold'em desarrollado en Godot con el lenguaje GDScript. Para esta primera entrega se pedía usar estructuras de datos lineales, y la idea desde el principio fue que cada una se usara porque realmente resolvía un problema del juego, y no solo porque se podía usar.

Se utilizaron tres estructuras: una **cola circular**, una **pila** y una **cola con prioridad**. En este documento se explica, para cada una, qué es, en qué parte del juego se usa, por qué encaja con lo que el juego necesita y qué alternativa se podía haber usado. Cuando una alternativa también habría funcionado, se dice abiertamente.

2. Cola circular: el orden de los turnos

2.1. Qué es

Una cola es una fila en la que se atiende primero a quien llegó primero. Una cola circular es una cola en la que, al llegar al final, se vuelve al principio. Se implementó con un arreglo de tamaño fijo, donde los índices "dan la vuelta" con el operador módulo.

2.2. Dónde se usa

Se usa para llevar el orden de los turnos de los jugadores en la mesa. El jugador al que le toca jugar es el que está al frente de la cola. Cuando termina su acción, sea *fold*, *check*, *raise* o *all-in*, se saca del frente y se vuelve a meter al final, y así el turno pasa al siguiente. También se usa para mover el botón del dealer, que avanza una posición en cada mano.

Los jugadores nunca se sacan de la cola. Un jugador que hace *fold* solo deja de actuar durante esa mano, y uno que se queda sin fichas se marca como eliminado, y en ambos casos simplemente se le salta el turno. Esto se maneja con banderas del jugador (`en_mano`, `puede_actuar` y `en_partida`), de modo que la cola se encarga solo del orden y las banderas del estado de cada jugador.

2.3. Por qué encaja

En una mesa de póker todos los jugadores son primeros y últimos en algún momento. Cuando juega el último, el turno vuelve al primero, y ese comportamiento cíclico es justamente el de una cola circular. Además:

- El número máximo de jugadores es conocido (nueve), por lo que un arreglo de tamaño fijo es suficiente.
- Pasar el turno solo mueve un marcador, sin desplazar a los demás jugadores.
- El orden de la mesa se conserva siempre, incluso cuando alguien se retira o es eliminado.

2.4. Qué alternativa había

La alternativa más simple era un arreglo con los jugadores y una variable que indica de quién es el turno, que avanza con el operador módulo. Funciona de forma casi idéntica a la cola circular, ya que en ambos casos hay un arreglo de tamaño fijo y una posición que da la vuelta. Se eligió encapsularlo como cola circular para que el resto del juego solo use operaciones como encolar, desencolar y rotar, sin manejar índices ni módulos directamente, lo que hace el código más claro y reduce los errores.

Otra opción era una lista circular enlazada, que permite eliminar jugadores del medio con más facilidad. Se descartó porque, al manejar la eliminación con banderas, no hace falta sacar a nadie de la estructura, y así la cola conserva solo las operaciones de encolar y desencolar.

2.5. Complejidad algorítmica

Las operaciones principales de la cola circular cuestan lo mismo sin importar cuántos jugadores haya. Como se implementó con un arreglo de tamaño fijo, pasar el turno solo mueve un marcador y no obliga a desplazar a los demás jugadores. Si se hubiera usado un arreglo normal sacando siempre el primer elemento, cada operación costaría $O(n)$, porque habría que correr todos los elementos una posición.

Operación	**Costo**
Encolar	$O(1)$
Desencolar	$O(1)$
Ver el frente y rotar	$O(1)$
Eliminar a alguien del medio	$O(n)$

La operación débil de una cola es eliminar a alguien que está en el medio, porque hay que sacar y volver a meter a todos los demás. Para evitarlo, en este proyecto no se elimina a nadie de la cola. Se marca al jugador con una bandera (`en_partida`, `en_mano` o `puede_actuar`), y esa marca cuesta $O(1)$. El precio es que los jugadores retirados o eliminados siguen en la cola y hay que saltarlos, lo que en el peor caso cuesta $O(n)$ por turno. Como hay a lo sumo 9 jugadores, ese costo es despreciable.

3. Pila: el mazo de cartas

3.1. Qué es

Una pila es una estructura en la que los elementos entran y salen por el mismo extremo, de modo que el último en entrar es el primero en salir. Se implementó con un arreglo de 52 posiciones y un marcador que indica cuál es la carta de arriba.

3.2. Dónde se usa

Se usa para el mazo. Al comenzar cada mano se meten las 52 cartas en la pila y se barajan con el algoritmo de Fisher-Yates. Después se reparten las cartas sacándolas de arriba: dos para cada jugador, una quemada antes de cada ronda y las cartas de la mesa (*flop*, *turn* y *river*).

3.3. Por qué encaja

Un mazo físico solo permite tomar la carta de arriba, y no se puede sacar una del medio ni mirar las de abajo. La pila funciona igual, porque solo expone las operaciones de meter y sacar por un único extremo. Esa restricción también protege el juego, ya que desde el código no se puede acceder a una carta cualquiera del mazo.

3.4. Qué alternativa había

Una cola habría funcionado igual. Como el mazo está barajado, el orden de las cartas es aleatorio, y da lo mismo sacarlas por un extremo o por el otro, así que el resultado del juego sería equivalente. Se eligió la pila porque representa mejor cómo se usa un mazo real, con un único punto de acceso, mientras que una cola tiene dos extremos activos.

3.5. Complejidad algorítmica

Meter una carta y sacar la de arriba cuestan $O(1)$, porque solo se trabaja sobre un extremo del arreglo y nunca hay que desplazar las demás cartas. Llenar el mazo y barajarlo con el algoritmo de Fisher-Yates recorren las 52 cartas una vez, por lo que cuestan $O(n)$, y se hacen solo al comenzar cada mano. Repartir las cartas es entonces una serie de operaciones $O(1)$.

Operación	**Costo**
`push` (meter una carta)	$O(1)$
`pop` (sacar la de arriba)	$O(1)$
Llenar el mazo	$O(n)$
Barajar (Fisher-Yates)	$O(n)$

Aquí $n = 52$ cartas, un número fijo y pequeño.

4. Cola con prioridad: el showdown y los botes laterales

4.1. Qué es

Una cola normal atiende en orden de llegada. En una cola con prioridad, cada elemento tiene una prioridad y siempre sale primero el que la tiene más alta (o más baja, según se defina), sin importar cuándo llegó. Se implementó como una lista enlazada ordenada, donde cada elemento se inserta en su posición según su prioridad y el primero de la lista es siempre el que sale.

4.2. Dónde se usa

Se usa en dos momentos, ambos al terminar la mano.

1. El showdown. Cuando se revelan las cartas, a la mano de cada jugador que sigue en juego se le asigna un puntaje según su combinación (pareja, color, escalera, etc.). Los jugadores se meten a la cola con ese puntaje como prioridad, y sale primero el de mejor mano. Si el siguiente tiene el mismo puntaje, hay un empate y el bote se reparte entre ellos. Si tiene menos, se detiene.

2. Los botes laterales. Cuando un jugador hace *all-in* con menos fichas que los demás, solo puede ganar hasta lo que aportó, por lo que el bote se divide en varios. Los jugadores

se meten a la cola con lo que aportó cada uno como prioridad, y sale primero el que menos puso. A medida que salen jugadores se van formando los botes.

Por ejemplo, si A puso 50 (*all-in*), B puso 150 y C puso 150:

- Sale A (50) y se forma el **bote principal** de $50 \times 3 = 150$, en el que compiten los tres.
- Salen B y C, que pusieron 100 más cada uno, y se forma un **bote lateral** de $100 \times 2 = 200$, en el que compiten solo B y C.

Cada bote se reparte con el procedimiento del showdown entre los jugadores que tienen derecho a él.

4.3. Por qué encaja

En ambos casos el problema es procesar elementos por importancia y no por orden de llegada, que es exactamente lo que hace una cola con prioridad. En el showdown, ordenar por fuerza de mano es el núcleo del problema, y los empates se detectan solos porque quedan consecutivos. Sin la estructura, habría que buscar al mejor jugador con ciclos y comparaciones manuales.

Se implementó como lista enlazada ordenada para mantener una estructura lineal, y no como un *heap*, que conceptualmente es un árbol.

4.4. Qué alternativa había

En el showdown, la alternativa era recorrer a los jugadores buscando el máximo con ciclos y comparaciones. Funciona, pero hay que repetirla por cada bote y tratar los empates aparte.

En los botes laterales, el uso es defendible pero no era el único posible. También se podían calcular con comparaciones y ciclos que buscaran, en cada paso, el menor aporte. La cola con prioridad evita repetir esa búsqueda, porque el orden se resuelve una sola vez dentro de la estructura.

4.5. Complejidad algorítmica

Se implementó como una lista enlazada ordenada. Sacar al jugador de mayor prioridad y ver quién sigue cuestan $O(1)$, porque siempre está al principio de la lista. En cambio, insertar un elemento cuesta $O(n)$, porque hay que recorrer la lista hasta encontrar su posición. Meter a todos los jugadores cuesta entonces $O(n^2)$ en el peor caso.

Operación	**Costo**
Insertar	$O(n)$
Desencolar (sacar el primero)	$O(1)$
Ver el frente	$O(1)$

Un *heap* haría la inserción en $O(\log n)$, pero es una estructura de tipo árbol y no lineal, y el proyecto pide estructuras lineales. Además, con a lo sumo 9 jugadores la diferencia de velocidad no se nota, por lo que se prefirió la lista ordenada, que es más sencilla de entender e implementar.

5. Conclusión

El desarrollo del juego permitió ver la importancia y la aplicación de las estructuras de datos lineales en un problema real. La cola circular resolvió de forma natural el orden cíclico de los turnos, la pila representó el comportamiento de un mazo de cartas, y la cola con prioridad permitió procesar por importancia el showdown y los botes laterales. En cada caso se analizó también qué alternativas había, para elegir la estructura que mejor se adaptaba al problema y no solo la que se podía usar.

Para las próximas entregas se espera mejorar el juego tanto en la interfaz como en el código, y también incorporar e interpretar estructuras de datos no lineales.

6. Uso de inteligencia artificial

En este proyecto se utilizó de forma amplia una herramienta de inteligencia artificial (Claude, de Anthropic), y es importante decirlo con honestidad. La herramienta se usó para escribir buena parte del código en GDScript, en particular la lógica del juego (turnos, reparto de cartas, evaluación de manos, showdown y botes laterales), para crear las pruebas automáticas de ese código, para resolver dudas sobre Git y GitHub, y para ayudar con la redacción y el formato de este documento.

Sin embargo, el uso no consistió en pedir que se hiciera todo sin más. Fue un proceso de conversación en el que hubo que explicar bien lo que se quería, revisar lo que se proponía y tomar decisiones propias. Algunos ejemplos:

- Se discutió qué estructuras lineales tenían sentido para el juego y cuáles serían forzadas, y se evaluaron alternativas antes de decidir.

- Se cuestionaron propuestas de la herramienta. Por ejemplo, se propuso manejar el fold y la eliminación con banderas en cada jugador en lugar de sacarlos de la cola, y se preguntó si era realmente necesario usar una cola con prioridad en los botes laterales.
- Se pidió que cada concepto se explicara hasta entenderlo, y las explicaciones de este documento se escribieron a partir de esa comprensión.

También hay que reconocer lo que no se hizo. No se aprendió a fondo la sintaxis de GDScript ni se escribió a mano la implementación completa, y se delegó esa parte en la herramienta. Lo que sí se buscó fue entender la lógica del programa, saber dar instrucciones precisas y poder explicar por qué cada estructura de datos se usa donde se usa. Se considera que esto es parecido a trabajar con un programador que ejecuta lo que se le pide, y que la responsabilidad de entender el diseño y de justificarlo sigue siendo de quienes entregan el trabajo.